Formal specification and verification of a trash-bin system

2IX20 Software Specification, 2026-2027 - Assignment 2

Verification date: 16 October 2026. Team members: **[TO COMPLETE: two names and student numbers]**.

Scope and model assumptions

This report covers sections 4.1-4.4, 5.1-5.4 and the reflection required by section 6.2. The supplied bin, user and server processes remain the environment. The controller and truck processes implement the protocol. Added authorization-history variables observe grants and opening commands without changing the user's choices. The supplied `trash_data.tla`, including `ValidCard(user) == {user # 42}`, is unchanged in the production models.

Model checking exhausts each specified finite configuration; it does not prove correctness for every possible number of components. The formulas quantify over all instances. The general model allows any positive numbers of bins, users and trucks, but unconditional user progress is impossible for a permanently invalid card, such as user 42. None of the required configurations contains user 42.

4.1 Main control: review and final design

The reviewed single-bin controller reads the oldest scan, asks the server to validate its user, and checks that the reply matches that user. It grants access only after a positive validation, sufficient capacity for `MaxUserTrash`, and completion of any collection. Reserving space for the maximum permitted deposit is conservative: the controller need not inspect the user's actual trash. The single user cannot overlap two deposits, so this reservation is sufficient until the trap opens.

The accepted-deposit sequence is: unlock and await acknowledgement; grant permission; await the door-closed sensor and completion of the close command; lock and await acknowledgement; open the trap; extend the ram; retract the ram; close the trap. Each command is acknowledged before the next is issued. The controller resets the door sensor after this cycle. This prevents a previous close event from satisfying the next deposit's wait. It never unlocks while trash remains in the top compartment.

The truck is requested when `Trash + MaxUserTrash > trashCapacity` after processing a scan/deposit, and only if its previous command is `emptied`. Thus collection is requested when another maximum-size deposit cannot fit, rather than after every deposit. The controller returns to scan processing while the truck travels or empties the bin. It sends a negative permission response when the bin is unavailable. A newly emptied bin stays reserved until the truck acknowledges completion; this also prevents interleaving a fresh user command with an unfinished collection handshake.

The earlier 4.3 authorization formula merely asked whether some valid user existed. That did not identify the opener. The final version records `grantedUser` after positive validation and `doorUser` when the user issues the opening command. It checks that an open door belongs to that validated grant. The grant is cleared once the door has been locked. The review also repaired missing verification scripts and replaced the incomplete single-bin handoff documentation with this combined report.

4.2 Truck: review and final design

The truck repeatedly waits for `request`, changes its status to `arrived`, then waits for an idle and safe bin. Before issuing `empty`, it requires a closed, locked outer door, closed trap, retracted ram, and no trash in the top compartment or uncompressed compartment. It changes status to `start_emptying`, issues the hardware command, and waits for `binCommand.command = "finished"`. Only then does it set `emptied` and accept another request.

Travel is a separate transition. There is no real-time duration in this untimed model; other processes may execute between truck steps. The truck changes commands, not the physical trash variables: only the supplied bin process actually empties the bin. This preserves the bin's assertions and makes `emptied` a completion acknowledgement.

4.3 Formal requirements and classification

Write `Fits == TrashFitsCapacity(MaxUserTrash)`. Here "full" means operationally full: insufficient space for another maximum-size permitted deposit. This includes physically full states and may trigger collection earlier. `~>` means leads to, `[]` means always, and `<>` means eventually. TLC checks the three state predicates below as invariants, so their required meaning is that they hold in every reachable state.

```
OuterDoorLocked == outerDoorLocked => ~outerDoorOpen

RamOuterDoor == ramExtended => (~outerDoorOpen /\ outerDoorLocked)

AuthorizedOpenOnly == outerDoorOpen =>
  /\ doorUser \in Users
  /\ doorUser = grantedUser
  /\ TRUE \in ValidCard(doorUser)

TrashEmptied == ~TrashFitsCapacity(MaxUserTrash)
               ~> TrashFitsCapacity(MaxUserTrash)

UserTrash == []<>(userTrash > 0) /\ []<>(userTrash = 0)

UserTrashDeposited == userTrash > 0 ~> userTrash = 0

TruckEmpties == truckCommand.command = "request"
                ~> truckCommand.command = "emptied"
```

Requirement	Kind	Reason for the formulation
OuterDoorLocked	Safety	A locked, open door is a finite bad state.
RamOuterDoor	Safety	Extension with an open or unlocked door is a finite bad state.
AuthorizedOpenOnly	Safety	The recorded opener must match an actual validated grant and a valid card.
TrashEmptied	Liveness	Every full occurrence must eventually be followed by an available bin.
UserTrash	Liveness	Both positive and zero trash must recur indefinitely, not just once.
UserTrashDeposited	Liveness	Every positive amount must eventually become zero; this happens only on deposit.
TruckEmpties	Liveness	Every request must eventually reach the truck's completion acknowledgement.

Authorization history is initialized to the sentinel 0, outside `Users`; an open door cannot be justified by the initial state or another user's grant. The required single-user run contains only a valid card, so it cannot on its own test denial. A separate all-invalid-card experiment checks the negative path. In the nondeterministic-card experiment, an access grant is recorded only in the branch that received `permission = TRUE`; potential validity alone does not cause a grant.

Deadlock is checked by TLC's built-in option. In addition, every required run checks `TypeOK`, `MessagesOK`, `CapacitiesRespected`, `TrapNotDestroyed`, `CapacityNotExceeded`, and every environment assertion. These detect malformed messages, overflowing queues, unsafe trap use and excessive trash independently of the seven named requirements.

4.4 Single-bin verification and fairness

The prescribed constants are `(NumBins, NumUsers, NumTrucks, MaxCapacity, MaxUserTrash) = (1,1,1,7,4)`. `Spec` includes weak fairness for every bin, user, truck, server and controller process, generated by their `fair process` declarations. Weak fairness requires an action that remains enabled to eventually execute. Each required step of an accepted deposit or requested collection eventually becomes continuously enabled. With one valid user, an unavailable bin eventually becomes available and the next scan can succeed. Strong fairness is unnecessary for this configuration.

All seven properties, all diagnostic invariants, assertions and deadlock checking pass together. The measurements are in the table in section 5.3; `verification/trash_single.log` is the full result. Four separate experiments remove all fairness by specifying `INIT Init` and `NEXT Next`, while keeping the same algorithm and constants. All four liveness requirements then fail:

No-fairness experiment	TLC counterexample
TrashEmptied	After deposits make the bin operationally full, the execution eventually stutters forever; the full state is never remedied.
UserTrash	Stuttering starts at state 5 while the user still has zero trash; positive trash never recurs.
UserTrashDeposited	The user obtains trash, reaches an enabled deposit, then the execution stutters before depositing.
TruckEmpties	The bin may even be physically emptied, but the execution stutters before the truck's `emptied` acknowledgement; the requested completion is never observed.

The complete traces are `verification/unfair_<property>.log`. These are infinite counterexamples with a finite prefix and a stuttering suffix, not deadlocks: enabled progress is simply never scheduled. Weak fairness removes them. Safety does not need fairness.

5.1 Multiple bins, users and trucks

The multiple-bin template's arrays and per-user permission queues are retained. The controller uses the bin and user from each FIFO scan, routes the server request and reply consistently, and sends permission only to that user's queue. Authorization histories are arrays indexed by bin. Each truck owns its own command slot.

Accepted deposit cycles are serialized to keep ownership simple. Other users can enqueue scans during a deposit. Truck processes run independently, and an outstanding collection never blocks the controller from handling scans and returning a negative response for an unavailable bin. Deposits in other available bins can proceed while a truck travels or empties a bin. The model does not claim to execute two accepted deposits simultaneously.

After compression, an operationally full bin is appended to `pendingBins`. At `ControlNext`, the controller assigns the oldest pending bin to the smallest-numbered available truck before processing another scan. It does not await the resulting collection in that atomic step. The FIFO collection queue prevents overtaking; deterministic truck selection reduces unnecessary branching. Several trucks may work on different bins at once. No equal-utilization requirement is imposed on trucks.

An assigned or queued bin is unavailable for access. `CollectionExclusive` checks that each pending bin occurs once, no queued bin is already assigned, and active trucks never share a bin. A truck retains its bin identifier through the request-to-completion handshake. Its indexed hardware waits are the same as in the single-bin protocol.

To prevent user starvation, `admissionQueue[b]` stores valid declined users in FIFO order. A retry keeps the user's position. When capacity is available and collection has completed, only the head user may receive a grant; a successful grant removes that ticket. Later users receive negative responses and join the tail, without duplicate entries. A user changing bins cancels its old ticket, and a negative card validation cancels its ticket. `AdmissionOK` checks bounded queue lengths, no duplicates within a bin, and at most one bin reservation per user. This preserves prompt negative responses while retaining a fair admission order across retries.

All processes use weak fairness. The label `UserAwaitClosedDoor:+` additionally imposes strong fairness only on that acknowledgement action. With several users, other commands can make the shared `finished` flag intermittently available. This localized fairness condition prevents a user who has deposited from waiting forever before generating new trash. Whole-process strong fairness is unnecessary. The supplied user's choices and hardware commands are unchanged.

Reproduced deadlock and repair

The pre-repair model reached a real deadlock in the Bins configuration, recorded in `verification/deadlock_before_bins.log`. At final state 164, compressed trash is `<<0, 2>>`, `pendingBins = <<2>>`, the truck is idle (`emptied`), the user awaits permission for bin 2, and the controller is at `ControlAwaitBinReady`. Both bin processes and the server are also waiting. A truck is available, but no transition can execute.

That label attempted to dispatch a truck and await readiness in the same PlusCal atomic step. An `await` guards the whole step; it cannot release the dispatcher so a truck can perform the collection needed to satisfy the wait. Adding

process fairness cannot enable a disabled action. The final controller dispatches at `ControlNext`, returns a negative permission for an unavailable bin, and continues servicing scans. Admission tickets preserve user order without blocking dispatch. TLC now checks the entire repaired state graph with deadlock checking enabled for all three required configurations.

5.2 Generalized requirements

Every single-bin property is generalized over the relevant instances. The same safety/liveness classification and the same meaning of operational fullness apply.

```
OuterDoorLocked == \A b \in Bins:
  outerDoorLocked[b] => ~outerDoorOpen[b]

RamOuterDoor == \A b \in Bins:
  ramExtended[b] => (~outerDoorOpen[b] /\ outerDoorLocked[b])

AuthorizedOpenOnly == \A b \in Bins:
  outerDoorOpen[b] =>
    /\ doorUser[b] \in Users
    /\ doorUser[b] = grantedUser[b]
    /\ TRUE \in ValidCard(doorUser[b])

TrashEmptied == \A b \in Bins:
  ~TrashFitsCapacity(b, MaxUserTrash) ~> TrashFitsCapacity(b, MaxUserTrash)

UserTrash == \A u \in Users:
  /\ []<>(userTrash[u] > 0)
  /\ []<>(userTrash[u] = 0)

UserTrashDeposited == \A u \in Users:
  userTrash[u] > 0 ~> userTrash[u] = 0

TruckEmpties == \A b \in Bins: \A t \in Trucks:
  (truckCommand[t].bin = b /\ truckCommand[t].command = "request")
  ~> (truckCommand[t].bin = b /\ truckCommand[t].command = "emptied")
```

The quantifier for `TruckEmpties` is outside leads-to. Thus the same bin and truck are tracked from request to completion. Completion of another bin, or the initial idle status of another truck, cannot discharge the request. Requiring its assigned truck to finish is slightly stronger than allowing any truck to finish, and matches the implemented ownership protocol.

5.3 Required verification results and scalability

TLC ran with the course's TLA+ 1.7.4 JAR (the executable reports TLC 2.19), Java 8, a 768 MB heap, one worker and fingerprint polynomial index 0. No symmetry reduction or state constraint was used for the required configurations. Times are measured process wall times, including startup and temporal checking; concurrent machine activity can affect them. Input hashes, commands and exact counts are recorded beside each log.

Configuration (B,U,T,C,M)	Distinct states	Wall time (s)	Outcome
Single (1,1,1,7,4)	5,286	8.584	All checks pass
Bins (2,1,1,3,2)	31,364	41.653	All checks pass
Users (1,2,1,3,2)	88,084	172.835	All checks pass
Trucks (2,1,2,2,1)	38,550	66.753	All checks pass

The additional complete Users safety/collection run visited 88,084 distinct states in 58.015 seconds. B, U and T are component counts; C is MaxCapacity and M is MaxUserTrash. Distinct-state counts do not include the multiplicity of temporal-checking branches.

Check	Single	Bins	Users	Trucks
No deadlock	Pass	Pass	Pass	Pass

Check	Single	Bins	Users	Trucks
OuterDoorLocked	Pass	Pass	Pass	Pass
RamOuterDoor	Pass	Pass	Pass	Pass
AuthorizedOpenOnly	Pass	Pass	Pass	Pass
TrashEmptied	Pass	Pass	Pass	Pass
UserTrash	Pass	Pass	Pass	Pass
UserTrashDeposited	Pass	Pass	Pass	Pass
TruckEmpties	Pass	Pass	Pass	Pass
All diagnostic invariants and assertions	Pass	Pass	Pass	Pass

All required properties are checked together in each primary run, including Users. `multiple_users_other.cfg` also exhausts the same Users graph with every invariant, deadlock checking and the two collection properties enabled. No required main-model violation is classified as an expected outcome: each main run requires TLC to exit successfully with no error.

Every primary configuration finishes within five minutes. Serializing accepted deposits, allowing only one server exchange at a time, using bounded FIFO queues and choosing the smallest idle truck limit branching. The model retains the original environmental choices. Strong fairness is limited to the intermittent close-acknowledgement action instead of every controller and user action. The Users graph contains 88,084 distinct states; its combined safety/liveness run completes in under three minutes on the measured machine. No state constraints, disabled deadlock checks or weakened requirement formulas are used to obtain these results. Larger configurations may still require substantially more resources.

5.4 Behaviour investigations

Multiple users and starvation

Both `UserTrash` and `UserTrashDeposited` now hold for Users. FIFO scan processing alone would not guarantee this: a denied request leaves the scan queue and its retry goes to the back, allowing the same user to lose repeatedly. The final model retains a separate admission ticket across negative capacity responses. Other users cannot consume the next opening ahead of the head ticket. With the required single-bin Users configuration and always-valid cards, an earlier ticket eventually succeeds, and every waiting user advances. Truck collection remains independent of successful card scans.

A separate scheduling issue remains relevant to fairness. After a deposit, a user awaits `binCommand.command = "finished"`. Other commands can repeatedly replace that shared flag, making this action enabled infinitely often without leaving it continuously enabled. Weak process fairness alone permits the user to remain at this acknowledgement with zero trash forever. The `users_weak_UserTrash` experiment removes only the additional label-level fairness by using `ProcessWeakSpec`; TLC reports this counterexample. Thus infinitely often having positive trash needs the stronger acknowledgement fairness. It is a liveness violation, not a deadlock.

Under the same process-weak specification, `UserTrashDeposited` passes: the admission queue ensures that a user who already has trash is eventually admitted. The problematic close acknowledgement occurs after the user's amount has already become zero. The final `Spec` adds `SF_vars(UserAwaitClosedDoor(u))` for each user, and both requirements then pass together in the unrestricted Users graph. No restricted replay witness is used for the final verification. Strong fairness of all processes would also suffice, but is more than is needed for this model.

Additional bins or trucks alone would not supply admission ordering or acknowledgement fairness. The model addresses those issues explicitly. The reported checks cover the three prescribed configurations, not every combination of arbitrary component counts.

Nondeterministically declined cards

The isolated `trash_scenarios` wrapper overrides `ValidCard` with `AnyCard(user) == {TRUE, FALSE}` through the TLC configuration; the production helper is unchanged. The Bins configuration is used. Both user liveness properties fail: the server can select FALSE on every repeated validation of a user who already has trash. That user scans, receives a decline, and retries forever, never depositing. Every participating process can be scheduled fairly. Strong process fairness would still not require the server's TRUE outcome, because it constrains the server process action, not each outcome of its internal choice.

All safety checks, deadlock checking, `TrashEmptied` and `TruckEmpties` pass in `declined_other`. An accepted deposit still follows the full safety protocol, and an already-requested truck does not depend on another successful card validation. `declined_UserTrash.log` and `declined_UserTrashDeposited.log` contain separate temporal counterexamples. The all-invalid-card test, `invalid_card_safety`, passes all safety checks: repeated negative replies never open a door. The two user progress properties are deliberately not asserted in that negative-path test.

Replacing the door sensor with `finished`

The experiment `trash_no_sensor` changes only the controller's post-grant wait to `await binCommand[currentScan.bin].command = "finished"`. It does not preserve the required behaviour. Immediately after unlock is acknowledged, `finished` is already true, even if the user has not opened or closed the door. In the TLC trace, permission is issued at state 11, the controller requests locking at state 12, and the bin locks at state 13. The user receives the positive permission at state 15 and issues an opening command at state 16. Executing that command fails the bin's `assert ~outerDoorLocked[self]`. The full trace is `sensor_finished_only.log`. The sensor supplies the missing evidence that the user has actually closed the door during this deposit. Command completion alone does not identify which command completed.

Maximum compressed trash with deposits of four and six

The isolated `trash_max_compressed` variant uses one bin of capacity 7 and two users. User 2 always generates 4 units and user 3 always generates 6. `MaxUserTrash = 6`. For this investigation only, admission uses the requesting user's actual amount and collection is disabled by not enqueuing bins. These changes are permitted by section 5.4. All capacity checks and the physical compression rule remain enabled; safety and deadlock checking are retained. The usual user/collection liveness properties are not required when collection is deliberately disabled.

```
BelowFive == trashCompressed[1] < 5
AtMostFive == trashCompressed[1] <= 5
```

`BelowFive` is intentionally false. Its counterexample deposits 6 first: the transient uncompressed total is 6, then compression leaves 3. It then deposits 4: the transient total is 3 + 4 = 7, and compression leaves 3 + 2 = 5. Thus five compressed units are reachable without exceeding capacity. Reversing the order gives 2 after the first compression and would require 2 + 6 = 8 for the next deposit, which is unsafe and denied.

The separate `AtMostFive` run exhausts the modified graph and passes, together with `CapacityNotExceeded`. This establishes maximality rather than merely showing reachability. Algebraically, a four-unit deposit is admissible only at compressed amount at most 3 and then adds 2, giving at most 5. A six-unit deposit is admissible only at amount at most 1 and then adds 3, giving at most 4. Starting from zero, no permitted compression can exceed 5. Once 5 is reached, neither fixed-size deposit fits.

Experiment	Visited states	Seconds	Observed result
Process fairness only: UserTrash	88,084	24.032	Temporal counterexample
Process fairness only: deposited	88,084	47.035	Pass
Declined cards: UserTrash	13,463	9.119	Temporal counterexample
Declined cards: deposited	14,373	9.006	Temporal counterexample
Declined cards: other checks	40,596	34.960	Pass
Always-invalid card: safety	48	5.951	Pass
Door wait uses finished only	116	5.964	Assertion failure

Experiment	Visited states	Seconds	Observed result
Compressed amount < 5	2,735	6.095	Invariant counterexample
Compressed amount <= 5	5,192	6.173	Pass

Counterexample searches may stop before exhausting the reachable graph. All-pass experiments exhaust their finite graphs. The four no-fairness experiments each visited 5,286 distinct states and produced the expected temporal violation; their timings are recorded in the corresponding JSON files.

6.2 Reflection and declaration of AI use

Technical reflection

The review exposed a weak authorization formula: the existence of any valid user did not establish that the actual opener had permission. Explicit grant and opener history made this relationship checkable. The main concurrency difficulties were reserving a bin until the truck acknowledges completion, avoiding duplicate truck assignments, and distinguishing a door-close event from an arbitrary command acknowledgement.

The investigations also distinguish request handling from starvation freedom and process scheduling fairness from fairness of nondeterministic choices. Counterexamples are necessary evidence here: a completed response to every scan does not imply eventual successful access. The deadlock repair also demonstrated why a command and a wait for its eventual effect must not share one atomic step. Admission tickets and localized acknowledgement fairness make both Users progress requirements hold. A separate upper-bound check was needed for the maximum-compression question; reaching five alone would not establish that six is impossible. Reproducible scripts and recorded hashes connect these claims to the exact submitted models.

Team reflection - requires the team's own factual account

- **[TO COMPLETE]** Member 1: name, student number, actual contributions and difficulties encountered.
- **[TO COMPLETE]** Member 2: name, student number, actual contributions and difficulties encountered.
- **[TO COMPLETE]** Other groups consulted, or an explicit statement that none were consulted.
- **[TO COMPLETE]** Any remaining questions or additional personal observations.

These statements have not been invented from the existing files. Complete them before submission.

AI-use declaration

This draft was prepared with OpenAI Codex. The following describes the observable assistance in this workspace; the team must review it and add any earlier AI use and its own acceptance/revision decisions.

Report section	Assistance and retained output
4.1-4.2	Reviewed the existing controller/truck; retained their protocol, added response matching and the collection-completion admission guard, and drafted the explanation.
4.3-4.4	Replaced the weak authorization formula with grant/opener history, retained the other requirement forms, ran TLC and no-fairness experiments, and drafted the results.
5.1-5.3	Implemented the controller/truck and FIFO collection queue; repaired the reproduced deadlock, added FIFO admission tickets and acknowledgement fairness, and reran all prescribed configurations.
5.4	Regenerated the isolated experiments for the repaired model, checked the need for acknowledgement fairness, inspected counterexamples, and verified the compressed-trash bound. The earlier replay witness was retired.
6.2 and packaging	Drafted the technical reflection and this disclosure, generated the PDF and ZIP; left personal contributions and group discussions for the team.

The model checker supplied the verification outcomes; AI-generated explanations do not substitute for those runs. The original supplied environment and data definitions were retained apart from observational authorization assignments and fairness declarations. The sensor replacement and nondeterministic-card policy are investigation variants, not production changes. **[TO COMPLETE: the team's own per-section kept/edited/rejected decisions and any additional tools used.]**